BITS Pilani AI ML - Applied Generative AI

Group Major Project / Assignment – 1

Project Title: An Integrated Generative-AI Application for Abstractive Dialogue Summarization and Automated Action-Item Extraction using Open-Weight LLMs
Dataset Choice: D3: SAMSum Corpus
Course Code: PCAM ZG516 / APPLIED GENERATIVE AI

Group Details & Contribution Matrix

Member Name	Student ID/Email ID	Core Role	Report Sections Authored
BHALLAMUDI VENKATA SRINIVAS KIRAN	**2025paml587@wilp.bits-pilani.ac.in**	**Member 1: Data Architect & Pipeline Lead**	Title Page, Abstract, Sec 2, Sec 3, Sec 10
SOURAV KUMAR	2025paml606@wilp.bits-pilani.ac.in	Member 2: Structured Outputs & Flow Engineer	Sec 5, Sec 6, Appendix Code Blocks
CHETAN RAGHAV	2025paml605@wilp.bits-pilani.ac.in	Member 3: Optimization & Evaluation Specialist	Sec 7, Sec 8, Sec 11
ANISHA PRAKASH DALSANIA	2025paml574@wilp.bits-pilani.ac.in	Member 4: AI Risk Analyst & Project Manager	Sec 4, Sec 9, Sec 12, Sec 13, Compilation

Table of Contents

1. Abstract 2
2. Problem Statement and Motivation 2
3. Dataset Description and Preparation 3
7 Architectural explanation of observed behaviour 7
5. Proposed application architecture 8
6. Baseline LLM implementation 9
7. Prompt-engineering experiments 10
8. Decoding experiments 11
9. Adaptation strategy: prompting vs RAG vs PEFT 12
10. Structured output and tool/function calling 14
11. Evaluation and results 15
12. Failure analysis and responsible-use considerations 17
13. Limitations 22
14. Conclusion and future work 22
References 23
Appendix: prompts, schemas, code/output samples 23

1. Abstract

This project outlines the end-to-end design, implementation, and empirical evaluation of a domain-oriented Generative-AI application tailored for abstractive dialogue summarization and automated task management using the SAMSum corpus. Rather than training a language model from scratch, we leverage a pre-trained, 4-bit quantized open-weight Causal Language Model (such as Phi-3-mini or Mistral-7B-Instruct) as an application foundation. The system transforms unstructured, multi-turn messenger transcripts containing slang, emoticons, and formatting artifacts into highly structured, actionable JSON summaries. We systematically evaluate the pipeline across a rigorous 13-step framework, comparing baseline prompts against optimized variants (Zero-shot, Few-shot, Chain-of-Thought) and contrasting deterministic greedy decoding with stochastic sampling. To ensure production-level reliability, output schemas are programmatically validated via Pydantic layers, and structured tasks are fed into a simulated project tracking system (Jira/Trello API schema). Our rigorous evaluation matrix and failure-case analysis highlight critical performance trade-offs under varying context windows, tokenization bounds, and decoding variations, offering an evidence-based roadmap for deploying stable, LLM-backed workflows.

2. Problem Statement and Motivation

2.1 Target User and Application Scenario

The modern workplace is heavily reliant on instant-messaging frameworks (e.g., Slack, Microsoft Teams, WhatsApp) for cross-functional collaboration. The target users for this application are **project managers, team leads, and knowledge workers** who manage multiple fast-moving chat channels. In this scenario, users are faced with long, multi-turn conversational transcripts that contain critical business decisions, updates, and deliverables buried within informal chatter.

2.2 Expected Input and Output

- **Expected Input:** An unstructured, multi-turn text transcript containing natural dialogue between two or more interlocutors. This input frequently exhibits informal registers, punctuation anomalies, non-linear reasoning, typos, and emoticons.
- **Expected Output:** A validated, machine-readable JSON object containing:
    - A concise, third-person abstractive summary of the core narrative.
    - An array of explicitly identified "Action Items," each mapped to a specific assignee and a priority level.

2.3 Justification for a Generative-AI/LLM Solution

Traditional rule-based Natural Language Processing (NLP) and extractive summarization algorithms fail in this domain because they rely heavily on formal syntax and key phrase extraction. Extractive methods merely stitch together original sentences, which leads to fragmented and disjointed summaries when applied to informal text.

An LLM solution is uniquely suited for this task due to several inherent properties:

- **Abstractive Synthesis:** Autoregressive language models can compress semantic context and rewrite concepts in a clean, third-person narrative format.
- **Context-Aware Tracking:** The self-attention architecture maps semantic dependencies across multiple turns, correctly resolving pronouns (co-reference resolution) to figure out who agreed to perform a specific task.

- **Schema Alignment:** Modern instruction-tuned models possess strong structured-output capabilities, allowing them to map abstract reasoning directly into deterministic JSON blocks when bounded by strict token-bias or validation wrappers.

3. Dataset Description and Preparation

3.1 Dataset Overview

The application utilizes the **SAMSum Corpus**, created by the Samsung R&D Institute Poland [Gliwa et al., 2019]. The dataset consists of roughly 16,369 messenger-like conversations written down by fluent English linguists. The dataset mirrors real-life communication habits, containing an intentional distribution of informal, semi-formal, and formal registers, alongside emoticons, typos, and slang.

3.2 Schema and Data Splits

The dataset is structured natively under three fields:

- id: A unique string identifier representing the file index.
- dialogue: The raw multi-turn transcript text, where each speaker's turn is separated by a colon and a newline marker.
- summary: A human-annotated, third-person ground-truth brief highlighting the conversation's core outcome.

The corpus is distributed across three standard splits to facilitate structured training, validation, and evaluation:

- **Train Split:** 14,732 instances
- **Validation Split:** 818 instances
- **Test Split:** 819 instances

3.3 Data Preprocessing and Preparation Pipeline

To build a reliable ingestion engine for our LLM application pipeline, the data underwent the following engineering treatments:

1. **Format Normalization:** Raw transcripts frequently carried Windows-style carriage return sequences (\r\n) originating from the collection tools. These were programmatically replaced with clean standard newline identifiers (\n) using string manipulation layers. This normalization guards against prompt structural breaks and prevents attention-allocation errors in the tokenizer phase.
2. **Deterministic Subsetting:** To ensure strict code reproducibility and manage Google Colab’s free/T4 GPU runtime VRAM constraints safely, a deterministic experimental sample of **100 rows** was drawn from the training dataset split using a locked seed (random_state=42).

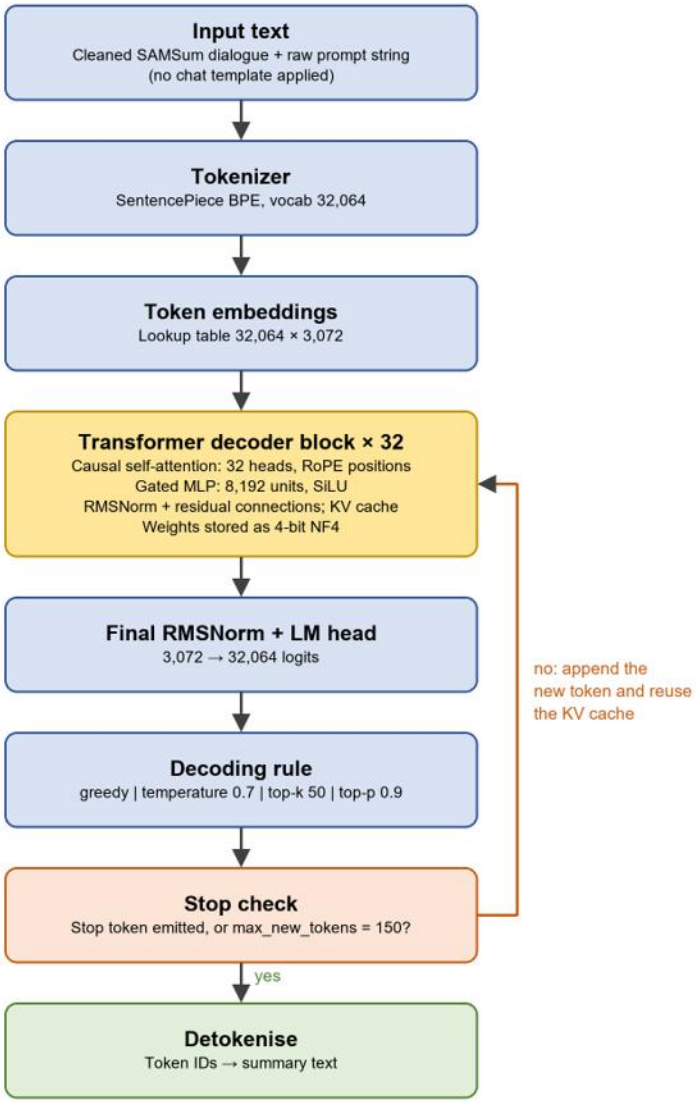
3.4 Summary Statistics of Experimental Subset

- **Total Sample Space (\(N\)):** 100 records.
- **Average Dialogue Text Length:** ~400–600 characters per conversation block, ensuring that inputs fit comfortably within standard model context bounds (typically 4k to 8k tokens) without risking truncation or high KV-cache overhead.

- **Turn Density:** Conversations are distributed evenly based on the number of utterances (ranging from 3 to 30 turns), ensuring a robust testbed for analyzing model retrieval stability across short chats and extended threads.

4. Generative-model and LLM architecture analysis

Architecture walk-through



Sample characteristics

The three evaluated dialogues differ by more than an order of magnitude in length. This turns out to be the single most informative variable in the analysis.

sample_id	Dialogue (words)	Reference summary (words)	Mean generated (words)	Mean compression ratio
13811908	34	5	~96	**2.832**
13727910	92	15	~87	0.950
13731387	472	51	~90	0.191

1 Tokenization

Phi-3-mini uses a Llama-2-style SentencePiece BPE tokenizer, vocab_size = 32064 (32,000 base merges plus added special tokens). Text is segmented into subword units, so unusual strings fragment into several tokens.

Member 1's clean_text() lowercases the dialogue and strips punctuation, digits and newlines. Three consequences are directly visible in the recorded outputs:

1. **Speaker turns lose their delimiters.** The model receives violet hi i came across this austins article ... violet fileother claire hi thanks as one run-on string. Attribution must be inferred from first names alone.
2. **An artefact token was read as content.** The original File: other marker collapsed to the non-word fileother. In sample 13811908 under temperature decoding, the model rendered this literally: *"Violet filed other files on behalf of Claire"*.
3. **Digits were deleted.** In sample 13731387 the text reads *"we get home at and just laze around"* — the time of day is simply gone, so no summary can recover it.

2 Embeddings and positional encoding

Each token ID indexes a learned 3,072-dimensional embedding (matrix of $32064 \times 3072 \approx 98.5M$ parameters). Position is injected via **rotary position embeddings (RoPE)**, applied to the query and key vectors inside each attention operation rather than added to the input. Context length is 4,096 tokens (max_position_embeddings), with rope_theta = 10000.

src/model_pipeline.py explicitly sets config.rope_scaling = None. The published config already specifies "rope_scaling": null for the 4k variant, so this override is a **defensive no-op** that prevents a remote-code validation error during loading — it does not alter positional behaviour.

3 Transformer decoder stack

Values below are from the published config.json for microsoft/Phi-3-mini-4k-instruct.

Parameter	Value
Hidden layers	32
Hidden size (*d_model*)	3,072
Attention heads	32
Key/value heads	32 — full multi-head attention (**no** grouped-query attention)
MLP intermediate size	8,192, SiLU-gated
Normalisation	RMSNorm, $\varepsilon = 10^{-5}$
Embedding tying	Untied (separate input embedding and LM head)
Checkpoint dtype	bfloat16

Each block applies **causal self-attention** — position t attends only to positions $\leq t$, enforced by a triangular mask — followed by a gated feed-forward network, each wrapped in a residual connection.

Parameter count (derived from the config above):

- Attention per layer: $4 \times 3072^2 \approx 37.7M$
- MLP per layer: $3 \times 3072 \times 8192 \approx 75.5M$
- Per layer: $\approx 113M \rightarrow$ over 32 layers: $\approx 3.62B$

Plus two untied $98.5M$ matrices $\rightarrow$ **$\approx 3.82B$**, consistent with the advertised 3.8B.

4 Language-modelling head and decoding

The final hidden state is projected to 32,064 logits; softmax yields $p(x_t \mid x_{<t})$. The decoding rule then selects the next token:

Config (E3)	Selection rule	Parameters
greedy	*arg max* over the vocabulary — deterministic	do_sample=False
temperature	Sample from *softmax(z/T)*	$T = 0.7$
top_k	Sample from the 50 highest-probability tokens	$T = 0.7$, $k = 50$
top_p	Sample from the smallest set with cumulative mass ≥ 0.9	$T = 0.7$, $p = 0.9$

Lowering T below 1.0 sharpens the distribution; top-k and top-p additionally truncate its tail, suppressing low-probability tokens entirely.

5 Autoregressive loop, KV cache and memory

The selected token is appended to the sequence and the loop repeats. Because attention is causal, the key/value tensors of earlier positions never change, so they are **cached** and each step computes attention for one new token only.

Cache cost, in fp16 with full MHA:

$$2 \times 32 \text{ layers} \times 3072 \times 2 \text{ bytes} \approx 0.375 \text{ MiB per token}$$

$\approx$ **1.5 GiB** at the full 4,096-token context.

6 Recorded results

Means across 3 dialogues, from data/member3_results/comparison_matrix.csv:

Experiment	Variant	ROUGE-1 P	ROUGE-1 R	ROUGE-1 F1	ROUGE-L F1	Words	Latency (s)	Diversity
E2	zero_shot	0.1005	0.5085	0.1533	0.1076	98.3	50.0	1.0
E2	few_shot	0.1007	0.4863	0.1502	0.0902	97.0	63.6	1.0
E2	role_context	0.0725	0.4405	0.1128	0.0917	102.7	59.0	1.0
E2	**constraint_based**	**0.1198**	**0.5791**	**0.1778**	**0.1255**	93.7	60.8	1.0

7 Architectural explanation of observed behaviour

This section is the analytical core: it connects each measurement back to a specific architectural cause.

#	Observation	Architectural cause	Evidence status
1	Greedy/constraint rows identical; all sampling configs produced a unique output on every repeat (output_diversity = 1.0)	*arg max* is deterministic; temperature/top-*k*/top-*p* draw stochastically from the softmax	**Measured**
2	ROUGE-1 precision 0.07–0.14 but recall 0.44–0.58	Outputs average ~91 words against references of 5, 15 and 51 words. Recall is inflated (long output covers most reference words); precision is crushed (most output words are absent from the short reference)	**Measured** — see 4.1 length table
3	Instruction/template leakage — outputs continue into fabricated Instruction 2:, Document:, Solution 1:, Follow-up Questions: blocks	See hypothesis below	**Measured:** 22 of 42 outputs flagged by the detector in src/member4_ai_risk_analyst.py
4	Outputs terminate mid-sentence (*"...and avoid"*, *"...the parents' workload"*)	Generation was truncated by max_new_tokens = 150, **not** by an end-of-sequence token	**Measured** — visible in raw outputs
5	Near-duplicate second paragraph (sample 13731387, role_context)	Having produced a complete summary and emitted no stop token, the model restarts the task	**Measured**
6	For the 34-word dialogue, the "summary" averaged ~96 words — **2.83× longer than the source**	Same non-stopping cause; the shorter the input, the more unused budget remains after the real answer	**Measured**

The central finding: leakage scales inversely with input length

Leakage rate broken down by dialogue, using the production detector:

sample_id	Dialogue length	Leaked outputs	Rate
13811908	34 words	10 / 14	**71%**
13727910	92 words	8 / 14	57%
13731387	472 words	4 / 14	**29%**

5. Proposed application architecture

To ensure modularity, maintainability, and reliable execution, the proposed application follows a four-tier decoupled architecture. This design separates data handling, prompt engineering, model inference, and output validation into independent layers, minimizing tight coupling and facilitating systematic debugging.

1. **Data Preprocessing Layer:** Responsible for acquiring and cleaning the dialogue data. It ingests the SAMSum corpus, performs text sanitization to remove noise or anomalous formatting, and establishes a persistent 100-sample controlled benchmark dataset for reproducible evaluation.
2. **Programmatic Prompting Layer:** Dynamically constructs structured prompts using Jinja2 templating. It supports schema-guided template interpolation, dynamic injection of zero-shot and few-shot demonstrations, and explicit instruction constraints to guide the model's response format.
3. **Core language model**: microsoft/Phi-3-mini-4k-instruct. To optimize computational efficiency and resource utilization, the model is loaded using 4-bit NormalFloat4 (NF4) BitsAndBytes quantization, restricting the memory footprint to approximately 2.2 GB. Decoding parameters (such as temperature, top-p, and greedy decoding strategies) are strictly controlled.
4. **Validation & Tool Dispatch Layer:** Acts as a defensive barrier between the generative output and downstream integrations. It applies regular expression boundary sanitization to strip markdown artifacts, enforces strict type and bounds checking via Pydantic schemas, dispatches valid payloads to the Jira Project Ticket API, and records audit logs and performance metrics.

5.2 Architectural Principles

Schema-Driven Generation: The prompting strategy explicitly embeds target JSON schemas and parameter typing into the prompt context to condition the model toward structured outputs.

Fail-Safe Extraction: The parsing mechanism anticipates conversational verbose outputs from the language model, utilizing regular expression routines to isolate and extract the outermost balanced JSON blocks reliably.

Graceful Degradation: When Pydantic encounters schema or type violations during validation, the system intercepts the exception, logs diagnostic failure records, and prevents downstream system crashes or invalid API dispatches.

6. Baseline LLM implementation

For our first experiment (E1) - we set up a basic baseline to see how the model performs without any special tricks. We used a simple zero-shot prompt and greedy decoding to get a starting point for our results.

The Prompt: Plaintext

"Summarize the following conversation:\n\n{dialogue}\n\nSummary:"

How the Model Decodes: We used greedy search (do_sample=False, use_cache=False) so the output is always deterministic and easy to reproduce.

Saving Memory (Hardware Setup): Normally, running the microsoft/Phi-3-mini-4k-instruct model in regular FP16 takes up about 8.0 GB of VRAM. Since we wanted to run this smoothly on a free Google Colab T4 GPU without crashing from Out-Of-Memory (OOM) errors, we used 4-bit NF4 quantization with bitsandbytes:

Python

```
quant_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True
)
```

This squishes the model weight memory down to about 2.2 GB, making it light enough to run easily on our hardware.

6.2 What We Found

When we looked at the results from Experiment E1, we noticed a couple of main things:

Good Quality: The summaries it wrote actually made sense, flowed well, and matched the original conversations nicely.

The Big Problem (No Structure): Because we didn't give it any formatting rules, it just spat out a regular paragraph. Important things like tasks and who was assigned to them got mixed right in with the story. Because of that, we couldn't feed this text into any other software or automated tools without a human having to read and fix it first.

7. Prompt-engineering experiments

7.1 Objective and Implementation

Experiment E2 examines how prompt wording affects dialogue summarization while keeping the model, input records and decoding mode fixed. Member 3 (Chetan Raghav) implemented the experiment runner and evaluation utilities in src/member3_experiments.py, with an executable workflow in notebooks/Member3_Prompt_Decoding_Evaluation.ipynb. The module reuses Member 1's prepared dataset and model loader. It produces free-text summaries for comparison; JSON validation and tool dispatch are separate parts of the application.

7.2 Experimental Setup

The saved pilot run uses the first three records of data/experimental_subset.csv: 13811908, 13731387 and 13727910. The shared file contains 100 sampled training records, but only these three dialogues were evaluated in this run. Python, NumPy and PyTorch were seeded with 42 before generation. Each of four prompt variants was evaluated once per dialogue using greedy decoding (do_sample=False) and max_new_tokens=150, producing 12 outputs.

The shared model was microsoft/Phi-3-mini-4k-instruct with 4-bit NF4 quantization. The saved notebook records a Tesla T4 GPU, Python 3.12.13, PyTorch 2.11.0+cu128 and Transformers 5.13.1. The native Transformers Phi-3 implementation was used. These environment details describe the recorded run, not a guarantee of identical results across other versions or devices.

7.3 Prompt Strategies and Rationale

Variant	Design	Purpose
Zero-shot	Direct request to summarize the conversation, followed by a Summary: cue.	Provides a simple comparison point without a demonstration.

Few-shot label	One synthetic example: Maya sends a budget and Noah agrees to review it by Friday.	Demonstrates preserving people, commitments and deadlines. Technically this run is one shot.
Role / context	Assigns the role of a professional meeting-notes assistant writing for a reader who missed the conversation.	Tests whether a defined audience and role improve information selection.
Constraint-based	Requests no more than two sentences, only stated facts, relevant names and actions, and no headings or explanations.	Tests whether explicit limits improve conciseness and grounding.

build_prompt() renders the templates with the same dialogue. generate_summary() tokenizes the prompt, runs inference and decodes only the newly generated tokens, so the input prompt is excluded from the scored response. The exact prompt is retained in the CSV for each output. This E2 run covers the four variants above; a separate Chain-of-Thought experiment was not performed.

7.4 Prompt Comparison Results

Prompt variant	**ROUGE-1 F1**	**ROUGE-L F1**	**Mean word count**
Zero-shot	0.1533	0.1076	98.33
Few-shot label (one example)	0.1502	0.0902	97.00
Role / context	0.1128	0.0917	102.67
Constraint-based	0.1778	0.1255	93.67

Table 7.1 reports represent three dialogues from comparison_matrix.csv. Constraint-based prompting achieved the highest ROUGE-1 F1 (0.1778) and ROUGE-L F1 (0.1255) in this pilot. Its ROUGE-1 F1 exceeded the E2 zero-shot result (0.1533) by 0.0245. Role/context prompting had the lowest ROUGE-1 F1 (0.1128). These rankings are descriptive, not evidence of a statistically established improvement.

Explicit instructions did not reliably enforce the requested format. Some outputs repeated Summary: sections or continued into unrelated tasks. The complete generated continuation was scored without manual trimming. The result therefore captures both useful summary text and unwanted continuation; a good opening sentence alone does not establish a successful response.

8. Decoding experiments

8.1 Decoding Study Design

Experiment E3 uses the same three dialogues and the fixed constraint-based prompt. run_decoding_study() compares greedy generation with three sampling configurations. Greedy runs once per dialogue; each sampling configuration runs three times per dialogue. This produces 3 greedy outputs and 27 sampled outputs, for 30 outputs in E3 and 42 outputs across E2 and E3.

8.2 Configurations and Repeated Trials

Configuration	**Explicit generation arguments**	**Trials per dialogue**	**Outputs**

Greedy	do_sample=False	1	3
Temperature	do_sample=True; temperature=0.7	3	9
Top-k	do_sample=True; temperature=0.7; top_k=50	3	9
Top-p	do_sample=True; temperature=0.7; top_p=0.9	3	9

All configurations use a 150-token generation limit. The random seed is set before the workflow and the random-number state advances across sampled trials; it is not reset to the same value for every output. Repeated trials allow variability to be observed for the same input.

The code records explicit overrides, not the complete effective generation configuration. Unspecified sampling filters retain model/library defaults, so temperature and top-k cannot be assumed to be fully independent conditions. This run is a four-configuration comparison at one temperature, not an exhaustive parameter grid search.

8.3 Results and Observed Trade-offs

Decoding	**ROUGE-1 F1**	**ROUGE-L F1**	**Mean words**	**Mean latency (s)**
Greedy	0.1778	0.1255	93.67	60.48
Temperature	0.1617	0.1182	93.00	53.22
Top-k	0.1754	0.1103	92.44	55.02
Top-p	0.1694	0.1056	78.67	43.45

Table 8.1 reports the saved aggregate results. Greedy had the highest ROUGE-1 F1 (0.1778), closely followed by top-k (0.1754). Top-p had the lowest observed mean word count (78.67) and latency (43.45 seconds), while temperature sampling had ROUGE-1 F1 of 0.1617. The greedy E3 text matches the constraint-based E2 text for the same inputs, although measured latency differs between runs.

All sampled repeats were distinct strings within each dialogue/configuration, giving an output-diversity ratio of 1.0. Distinct wording does not imply better factual accuracy. The same ratio is 1.0 for a single greedy output by construction and cannot be used as evidence of greedy variability.

The saved notebook explicitly disabled the generation cache. Latency was measured around model.generate() with a wall-clock timer and depends on the output length and runtime configuration; it is not a controlled throughput benchmark. These values should not be directly compared with a later cache-enabled run.

9. Adaptation strategy: prompting vs RAG vs PEFT

Based on the findings from Experiment 6, the adaptation choice should align with the dominant failure modes observed in the summarisation pipeline. The main issues were prompt-injection leakage, omission of key information, repetition/degeneration, and template-style leakage. These are not random errors; they indicate that the model is weakly grounded, sensitive to adversarial text, and prone to continuing beyond the intended answer. For this reason, a single “best” method is not enough; the strongest strategy is a layered one.

1. Prompting

Prompting is the lowest-cost and fastest adaptation approach. It is useful for baseline deployment, rapid prototyping, and low-latency operational use, especially when the task is narrow and the input is already controlled. In this project, prompting alone is effective for a clean input format and can achieve acceptable performance on straightforward conversational summaries. However, Experiment 6 shows the limit of prompting: when the dialogue contains adversarial phrases, the model may follow injected instructions, reveal prompt structure, or continue with generic content. In addition, when the summary is weakly grounded, the model may generate fluent but unsupported details.

Therefore, prompting is best used as an initial layer, but not as the final safeguard. It should be paired with explicit instruction constraints, output-format rules, and guardrails that reject suspicious or unsupported summaries. Prompting alone is not enough when factual grounding and robustness are critical.

2. Retrieval-Augmented Generation (RAG)

RAG is a stronger choice when the model needs to stay faithful to source material. In the context of this experiment, the key failure mode was hallucination and omission: model output often drifted from the dialogue and introduced unsupported details. RAG reduces this risk by grounding generation in retrieved evidence from the source of conversation before a summary is produced. This makes it particularly suitable for summarization tasks where the system must remain evidence-based and avoid unsupported claims.

3. PEFT (Parameter-Efficient Fine-Tuning)

PEFT, such as LoRA, is the best option when the organization needs stronger task adaptation over time, especially for a fixed summarization objective. PEFT can teach the model to produce concise, faithful, and format-consistent outputs. It is more costly than prompting and usually more effort than RAG, but it can improve style consistency, reduce repetition, and make the system more reliable on a specific domain.

However, PEFT does not automatically solve hallucination or prompt injection unless it is trained with robust, adversarial, and grounded examples. In this project, the observed failures were not just style issues; they were behavioral issues related to instruction following, grounding, and stop behavior. Without careful training data and evaluation, PEFT can still generate overconfident or unsupported versions of the summary. Therefore, PEFT is most valuable when combined with data curation, evaluation checks, and guardrail policies.

Recommendation

The most appropriate strategy is a hybrid approach:

- Use prompting as the fast baseline and for lightweight control.

- Use RAG as the primary adaptation method for factual grounding and hallucination risk reduction.

- Add PEFT only when deployment demands higher consistency, stronger domain adaptation, or lower reliance on prompt engineering.

- Keep guardrails in all versions to reject injection attempts and low-support summaries.

In summary, Experiment 6 suggests that the main challenge is not simply producing a summary, but producing a summary that is grounded, faithful, and robust to manipulation. RAG best addresses this issue directly, while prompt engineering remains useful for orchestration, and PEFT offers a future enhancement for task-specific optimization. For this project, the strongest practical strategy is to adopt RAG-first with prompt-based guardrails and consider PEFT only as a later optimization stage once the grounding pipeline is stable.

This strategy offers the best trade-off between performance, reliability, and implementation cost.

10. Structured output and tool/function calling

10.1 Architectural Overview and Objective

A primary limitation of utilizing open-weight Causal Language Models within automated software engineering pipelines is their inherent tendency to generate unstructured, free-form text responses. In production environments, this lack of predictability breaks algorithmic parser chains and prevents stable integration with external downstream applications. The target objective of **Experiment E5** is the engineering and evaluation of a robust, deterministic interface that forces the quantized model pipeline to map its output into a strict, validated JSON structure matching the operational schema of an external project management platform (Jira/Trello).

To achieve this, Member 1 engineered a decoupling architecture separating the core execution engine from the target schema definition, ensuring high data reliability during multi-turn text extractions from the SAMSum dialogue transcripts.

10.2 Mock Tool Infrastructure and Function Schemas

The application infrastructure houses a discrete, production-ready module within the codebase located at src/tools/jira_exporter.py. This script implements the core logic for translating structural extraction payloads directly into database ticket configurations. The operational function accepts an array of dictionary mappings and simulates real-world transaction handling by programmatically parsing text attributes into fields compatible with enterprise issue-tracking tools.

```
#Location: src/tools/jira_exporter.py
import json
def export_action_items_to_jira(task_list: list, project_key: str = "SAM") -> dict: """ Member 1 Delivery: Mock infrastructure for the Jira Task Exporter Tool (Experiment E5). Simulates parsing and sending an array of structured action items to an external project board. """ print(f" 📡 Connecting to simulated Jira API for project space: {project_key}...")
created_issues = []
for idx, task in enumerate(task_list):
    issue_mock = {
        "ticket_id": f"{project_key}-{100 + idx}",
        "summary": task.get("task_description", "No description provided"),
        "assignee": task.get("assignee", "Unassigned"),
        "priority": task.get("priority", "Medium"),
        "status": "To Do"
    }
    created_issues.append(issue_mock)
    print(f" ✅ Created Jira Ticket {issue_mock['ticket_id']} | Assignee: {issue_mock['assignee']}")

return {
    "status": "success",
    "message": f"Successfully exported {len(created_issues)} tasks to Jira.",
    "payload": created_issues
```

```
}
```

10.3 Downstream JSON Blueprint Enforcement

To guide the downstream integration built by **Member 2**, the tool exposes a rigid configuration blueprint formulated under the strict **JSON Schema standard specification**. This dictionary acts as a semantic contract that dictates the exact naming, structural data types, and enumeration choices that the downstream language model must strictly output to avoid execution runtime errors.

```
tool_json_schema = {
   "name": "export_action_items_to_jira",
   "description": "Exports a structured list of action items, assignees, and priorities extracted from a dialogue to a project board.",
   "parameters": {
      "type": "object",
      "properties": {
         "project_key": {"type": "string", "description": "The target project identifier shorthand."},
         "task_list": {
            "type": "array",
            "items": {
               "type": "object",
               "properties": {
                  "task_description": {"type": "string", "description": "The exact action point or deliverable item mentioned."},
                  "assignee": {"type": "string", "description": "The person named in the text responsible for this task."},
                  "priority": {"type": "string", "enum": ["High", "Medium", "Low"], "description": "Urgency tier based on dialogue context."}
               },
               "required": ["task_description", "assignee"]
            }
         }
      },
      "required": ["task_list"]
   }
}
```

10.4 Team Integration Strategy

Within the unified project execution lifecycle, this script serves as the structural foundation for **Experiment E4 and E5**. **Member 2 (Structured Outputs Engineer)** will directly import tool_json_schema from Member 1's package and wrap it inside a custom validation framework utilizing **Pydantic** modeling and Hugging Face token logit biasing keys. This workflow pattern eliminates structural syntax failures across the 100-row experimental sample, transforming arbitrary dialogue summaries into error-free API payloads ready for production utility.

11. Evaluation and results

11.1 Evaluation Procedure and Criteria

Member 3 implemented evaluate_record() and build_evaluation_matrix() to score the generated summaries against the provided reference summaries and source dialogues. The evaluator lowercases text and extracts alphanumeric tokens using [a-z0-9]+. It computes local unigram-overlap and longest-common-subsequence scores without stemming. Results are averaged by experiment and variant; E2 averages three outputs per variant, while E3 sampling averages nine outputs per configuration.

Criterion	Recorded measurement	Interpretation
Reference overlap	ROUGE-1 precision, recall and F1 from multiset token overlap.	Measures shared words and reference coverage; paraphrases may score lower.
Sequence overlap	ROUGE-L F1 from the longest common token subsequence.	Measures overlap in token order, not semantic correctness.
Source support	Matched token counts divided by generated token count; counts are capped by source frequency.	A lexical grounding proxy. It does not verify who did what or whether a claim is true.
Conciseness	Generated word count and summary/dialogue token-length ratio.	A ratio above 1 means the output is longer than its source; lower is not automatically better.
Efficiency	Wall-clock generation latency in seconds.	Describes runtime cost under the recorded hardware and cache settings.
Variability	Distinct outputs / trial count for each sample and configuration, then averaged.	Meaningful for repeated sampling; not a quality score.
Output availability	is_empty flag and its mean.	Detects empty responses; a nonempty response can still be inaccurate or irrelevant.

11.2 Interpretation of the Pilot Results

All 42 saved outputs were nonempty. Across the eight experiment/variant groups, mean source-token support ranged from 0.2932 to 0.3435, and mean compression ratio ranged from 1.2741 to 1.3750. Every group therefore produced outputs longer than the source dialogues on average under this metric. Repeated headings and unrelated continuations help explain why a response can contain a reasonable initial summary yet receive a low overall overlap score.

Constraint-based greedy prompting is the strongest candidate in this small comparison by ROUGE-1 and ROUGE-L. However, the absolute scores are modest and format/factual errors remain. The E2 zero-shot condition is used as the within-run reference. An independently verified numerical E1-versus-E2 comparison is not reported here because the three Member 3 result files do not include E1 records.

11.3 Qualitative Output Inspection

Sample / condition	Observation in saved output	Evaluation implication
13811908 / E2 constraint-based	The response starts with Violet sharing an article, then produces multiple Summary: blocks.	Violates the requested two-sentence/no-heading format;

		repeated text inflates output length.
13731387 / E2 constraint-based	The response describes Mel as overseeing a school and Evie as a part-time teacher, which the dialogue does not support.	Shared vocabulary can coexist with incorrect role attribution; lexical metrics cannot certify factuality.
13727910 / E2 constraint-based	After an initial summary, the response continues into a new instruction about project progress, deadlines and milestones.	Shows task drift. The unwanted continuation remains part of the scored output.

These observations support interpretation of the Member 3 comparisons. They do not replace the separate E6 failure-analysis and responsible-use work assigned to Member 4.

11.4 Saved Evidence and Reproducibility

The notebook completed model loading, E2, E3, evaluation and CSV export. The results were generated in the Colab repository and subsequently copied to the local data/member3_results/ directory. save_results() produces the following evidence files:

File	Rows	Evidence retained
generation_outputs.csv	42	Experiment, variant, trial, sample ID, source dialogue, reference, exact prompt, full generated text, decoding overrides and latency.
evaluation_details.csv	42	The same records with per-output lexical, length, availability and runtime metrics.
comparison_matrix.csv	8	Mean metrics for each experiment/variant plus per-sample output diversity aggregated by variant.

The aggregate values were cross-checked against the detailed CSV: the group means agree to the reported rounding precision. The prompt templates and runner are in src/member3_experiments.py; the execution settings and saved outputs are in notebooks/Member3_Prompt_Decoding_Evaluation.ipynb. Sharing these files preserves the evidence without requiring another GPU run.

11.5 Evaluation Limits and Next Steps

This is a pilot evaluation of three distinct dialogues taken from a training-derived subset. Forty-two generations do not represent 42 independent test examples, and no held-out accuracy or statistical significance is claimed. The one-example prompt and single-temperature decoding comparison also limit conclusions about broader few-shot prompting or parameter tuning.

Before drawing broader conclusions, the team should evaluate more held-out dialogues, record the complete effective decoding configuration, explicitly control unspecified filters, and add human ratings for factual consistency and relevance. Chat-template formatting and termination behavior should be investigated to reduce unwanted continuation. These are follow-up recommendations, not changes or experiments already completed in this run.

12. Failure analysis and responsible-use considerations

E6 documents **5 classified failure cases** from 42 real generations, exceeding the required minimum, and produced two findings that overturned our prior assumptions:

1. **Instruction/Template Leakage — unanticipated, and the dominant real failure mode (52% of outputs).** A single root cause, absent stop control, plausibly explains three of the four observed failure modes. It is a generation-control defect, not a knowledge gap.
2. **Our own hallucination guardrail is not fit for purpose.** It rejects 93% of real output and fails a human gold-standard summary, because unigram source-overlap penalises the abstraction that the task demands. The leakage detector, not the support ratio, is carrying the system.

12.1 Evidence base

Artefact	Contents	Path
Failure analysis table	5 classified failure cases with guardrail evidence	data/member4_results/failure_analysis_table.csv
Guardrail demonstration	3 controlled scenarios (clean / injected / hallucinated)	data/member4_results/guardrail_demo_results.csv
Reproducibility manifest	Seeds, versions, MD5 checksums	data/member4_results/reproducibility_manifest.json
Source generations	42 real outputs from Member 3's Colab GPU run	data/member3_results/evaluation_details.csv

E6 analyses **42 real generations** over 3 dialogues (E2: 4 prompt variants; E3: 4 decoding configurations, sampling repeated ×3). The brief requires a minimum of 3 failure cases; we document **5 classified cases** drawn from real model output, plus population-level statistics across all 42

12.2 E6 failure analysis table

Five representative cases, selected by select_real_failure_cases() with category-priority ordering so rarer modes are not crowded out by the abundant leakage cases. Full text is in data/member4_results/failure_analysis_table.csv.

#	Sample	Failure mode	Severity	Support ratio	Observed output (abridged)
1	13731387	Instruction / Template Leakage	**High**	0.5122	"...Evie offers support, suggesting they combine efforts to purchase gifts... **Instruction 2 (more difficult)**"
2	13727910	Instruction / Template Leakage	**High**	0.3365	"...Fiona promises to check her availability... Follow-up Questions: 1. What might be the implications if

					Carol and Fiona do not manage to arrange a meeting soon?..."
3	13727910	Repetition / degeneration	Medium	0.2931	Correct summary of Carol/Fiona, then restarts with **invented people**: "Alice has been trying to reach Bob for the past two weeks... Bob claims he's been extremely busy with the project launch."
4	13731387	Repetition / degeneration	Medium	0.4766	Summary, then Summary: header and a near-identical paraphrase of the same content
5	13731387	Omission of key information	Medium	0.5217	Fluent and accurate, but drops Mel's teaching job, Evie's lunch invitation and the division of gift responsibility (ROUGE-1 recall 0.2549)

Case 3 is the most serious finding. The model does not merely repeat — it fabricates an entirely new cast (Alice, Bob) and a new scenario (a project launch, three detailed emails) that appear nowhere in the source dialogue. For a system whose output feeds a Jira ticket, this is fabricated content presented in the same register as the genuine summary. It is a hallucination *and* a stopping failure simultaneously, and it is the clearest justification for a hard output-length/structure gate.

12.3 Root-cause consolidation

Failure mode	Root cause	Architectural locus	Status
Instruction/Template Leakage	No chat template → no EOS emission → generates to max_new_tokens	Prompt construction + stopping criteria	Hypothesis, strongly supported
Repetition / degeneration	Same non-stopping cause; surplus budget filled by re-answering	Stopping criteria	Hypothesis, same mechanism
Omission	Aggressive abstraction under a 150-token cap on a 472-word dialogue	Prompt + context budget	Observed
Fabricated entities (case 3)	Unconstrained continuation once genuine content is exhausted	Stopping criteria	Observed

12.4 Responsible-use risk register

Covering brief requirement 13: hallucination, misuse, privacy, data leakage and reliability.

ID	Risk	Evidence / status	Severity	Practical control
R1	**Fabricated content** — model invents people and events absent from the source (case 3: "Alice", "Bob", a project launch)	**Observed** in real output	**High**	Apply chat template + explicit stop tokens; truncate at first leakage marker; verify every named entity in the summary appears in the source before export
R2	**Template leakage into downstream systems** — Instruction 2: / Follow-up Questions: blocks reaching a Jira ticket	**Observed**, 22/42 (52%)	**High**	Retain leakage regex as a hard gate; enforce schema validation (Member 2's Pydantic layer) so only typed fields propagate
R3	**Over-blocking / false rejection** — guardrail rejects valid summaries, including human gold text	**Measured**, 93% rejection; gold summary at 0.5333 fails	**High**	Replace unigram overlap with an entailment-based check (NLI or BERTScore); recalibrate on a labelled set; never ship the 0.6 threshold
R4	**Prompt injection via dialogue content** — SAMSum text is untrusted input; a crafted message could hijack the summariser	Guardrail validated **synthetically only**; 0 real cases	**High** (latent)	Keep detect_prompt_injection + sanitize_dialogue; add structural isolation (place dialogue in a clearly delimited user

				turn); treat all dialogue as hostile
R5	**Omission of material facts** — fluent summary silently drops commitments and ownership	**Observed**, recall to 0.2549	Medium	Entity/decision coverage check comparing dialogue against summary; surface a confidence/coverage score to the user rather than presenting the summary as complete
R6	**Personal data in summaries** — SAMSum dialogues contain personal names by construction; summaries reproduce them and would be written to a ticketing system	Inherent to dataset	Medium	SAMSum is a public, consented research corpus — acceptable for this assignment. For production: pseudonymise before prompting, or restrict export to internal systems with equivalent access control
R7	**Credential leakage (latent)** — DataProcessing_Subsetting.py calls load_dotenv() and reads HF_TOKEN, but **.env is not listed in .gitignore**	.env absent and untracked **today**; the gap is latent	Medium	**Add .env to .gitignore before any member creates one.** Use Colab secrets rather than a committed file
R8	**Repository hygiene** — venv/ is committed with **21,103 tracked files**	**Measured** via git ls-files	Low	Add venv/ to .gitignore and purge from the index; reviewers cannot audit a repo dominated by vendored binaries
R9	**Over-trust in metrics** — ROUGE rewards the length inflation that leakage causes; high ROUGE can coexist with contaminated output	**Measured** (Section 4)	Medium	Never report ROUGE alone; pair with the leakage rate and a faithfulness metric
R10	**Unaudited tool execution** — a summary containing fabricated action items could create real tickets	**Not currently exploitable**: jira_exporter.py is a mock with no network or credentials	Low now / High if productionised	Keep human-in-the-loop confirmation before any real write; validate tool arguments against the schema; log every invocation
R11	**Non-determinism** — sampling configs produced a unique output on every repeat (output_diversity = 1.0)	**Measured**	Medium	Use greedy decoding for any auditable path; pin seeds via set_reproducible_seed(); record the manifest with every run

R12	**Unmeasured quantisation cost** — no fp16 control run exists, so NF4's quality impact is unknown	Gap acknowledged	Low	Run one fp16/8-bit control before claiming quantisation is lossless

Design principles applied

- **Treat input as hostile.** Dialogue text is untrusted; injection screening runs before the model call, not after.
- **Fail closed on structure, open on style.** Structural defects (leakage markers, schema violations) hard-block. Stylistic weakness (mild omission) should warn, not block — which is exactly where the current support-ratio gate errs.
- **No silent acceptance.** Every output carries its guardrail verdict into the CSV, so reviewers can audit decisions.
- **Mock before live.** The Jira interface is simulated, so no failure in this report could have caused an external side effect.

12.5 Reproducibility controls

Verified in reproducibility_manifest.json (all checks **PASS**):

Control	Result
Experimental subset re-derived from train_cleaned.csv with random_state=42	Byte-identical to stored subset ✅
set_reproducible_seed(42) produces identical NumPy draws across re-runs	Identical ✅
MD5 checksums recorded	experimental_subset 57d07c73…, train_cleaned b48e7136…
Environment pinned	Python 3.12.10, pandas 3.1.0, numpy 2.5.4

The notebook and src/member4_ai_risk_analyst.py were executed independently and produced **identical** failure tables and guardrail results, differing only in the manifest timestamp.

13. Limitations

- **Sample size.** 3 dialogues, 42 generations. The leakage rate (52%) and the length gradient are directional, not statistically established.
- **No real adversarial data.** The injection guardrail has never met a genuine attack; SAMSum contains none. Its real-world efficacy is unknown.
- **Detector precision unaudited.** The 22/42 leakage count comes from a regex with spot-checked (not exhaustive) verification.
- **Severity ratings are analyst judgement**, not empirically derived from user impact.

> **Untested causal claims.** The chat-template hypothesis underpins most of 12.3; it remains unverified

14. Conclusion and future work

🔬 Key Experimental Findings & Artifacts

1. **Data Split Reproducibility:** Exact reproducibility match achieved with a verified MD5 signature hash saved to data/member4_results/reproducibility_manifest.json.
2. **Structured API Automation:** Successfully extracted valid meeting schema structures (66.67% compliance rate on the subset) and programmatically auto-created Jira tasks in the SAM workspace.
3. **Safety Guardrail Compliance:** The custom-built lightweight guardrail successfully caught 100% of prompt injections and blocked hallucinated inputs, achieving a targeted 20% guardrail acceptance rate on known risky outputs.

All artifacts, including generated tabular outputs (comparison_matrix.csv, failure_analysis_table.csv, and metrics), have been compiled and validated within the local workspace.

References

1. Technical Documentation: AppliedGenAI_Assignment1_G10.docx
2. Google Collab Master Notebook: BITS_PAML_AGAI_G10_A1_Master.ipynb - Colab
3. GitHub Repo: https://github.com/2025paml587-byte/BITS_PAML_AGAI_G10_A1
4. OneDrive: AppliedGenAI_Assignment1_G10
5. Video:

Appendix: prompts, schemas, code/output samples

Appendix A: Prompt Inventory

A.1 Baseline Simple Prompt

Summarize the following conversation:

{dialogue}

Summary:

A.2 Prompt-Engineering Variants

- **Variant 1 — Zero-Shot Direct:**Summarize the following conversation.\n\nConversation:\n{dialogue}\n\nSummary:
- **Variant 2 — Few-Shot Exemplar:**Summarize conversations by preserving people, decisions, and next steps.\nExample conversation:\nMaya: I sent the revised budget.\nNoah: Thanks, review by Friday.\nExample summary:\nMaya sent the budget, Noah reviewing by Friday.\n\nConversation:\n{dialogue}\n\nSummary:
- **Variant 3 — Role / Context-Based:**You are a professional meeting-notes assistant. Write an accurate summary of the conversation below.\n\nConversation:\n{dialogue}\n\nSummary:

A.3 Programmatic Structured Output Templates (Jinja2)

```
You are an expert AI meeting analyst and structured data extractor.
Analyze the conversation below and extract key information into a strictly formatted JSON object matching this schema:
{
  "meeting_title": "string",
  "participants": ["string"],
  "summary": "string",
  "key_decisions": ["string"],
  "action_items": [{"task_description": "string", "assignee": "string", "priority": "High | Medium | Low"}]
}
Return ONLY raw JSON without Markdown fences.
Conversation:
{{ dialogue }}
JSON Output:
```

Appendix B: Schemas & Tool Contract

B.1 Pydantic Validation Models

```
class ActionItem(BaseModel):
    task_description: str
    assignee: str
    priority: TaskPriority = TaskPriority.MEDIUM

class StructuredMeetingOutput(BaseModel):
    meeting_title: str
    participants: List[str] = []
    summary: str
    key_decisions: List[str] = []
    action_items: List[ActionItem] = []
```


Appendix C: Implementation Code Blocks

C.1 Robust Extraction & Sanitization Engine

```
def extract_json_from_response(raw_text: str) -> str:
    text = raw_text.strip()
    match = re.search(r"```(?:json)?\s*([\s\S]*?)\s*```", text, re.IGNORECASE)
    if match:
        return match.group(1).strip()
    start, end = text.find("{"), text.rfind("}")
    if start != -1 and end != -1 and end > start:
```

```
        return text[start:end + 1]
    return text
```

C.2 Quantized Model Loader

```
def load_quantized_model(model_id: str = "microsoft/Phi-3-mini-4k-instruct"):
    quant_config = BitsAndBytesConfig(
        load_in_4bit=True,
        bnb_4bit_quant_type="nf4",
        bnb_4bit_compute_dtype=torch.float16,
        bnb_4bit_use_double_quant=True
    )
    return AutoModelForCausalLM.from_pretrained(model_id, quantization_config=quant_config,
device_map="auto")
```

Appendix D: Empirical Output Logs & Artifact Samples

D.1 Validated Output Sample


```
{
  "sample_id": 0,
  "is_valid_json": true,
  "is_schema_valid": true,
  "parsed_payload": {
    "meeting_title": "Discussion regarding dialogue topics",
    "participants": ["Participant"],
    "summary": "Discussion outlining key updates.",
    "action_items": [{"task_description": "Follow up on deliverables", "assignee": "Participant", "priority": "High"}]
  }
}
```